ИТЕРАЦИЯ 5

Проектирование базы данных

Сколько хранилищ и почему, SQL или NoSQL, схема из диаграммы классов и **проверка на рабочей базе PostgreSQL 16**

УЧАСТНИКИ

Бобков А.Н.

Герасимов Д.С.

Соколов В.И.

Шохин А.А.

СОДЕРЖАНИЕ ИТЕРАЦИИ

Четыре вопроса пятой итерации

01

Сколько баз данных

Два хранилища с разным назначением: PostgreSQL и Redis. Критерий — характер данных

02

SQL или NoSQL

Пять аргументов за реляционную модель и осознанные NoSQL-островки на JSONB

03

Схема из классов

14 таблиц по ORM-маппингу проектной диаграммы классов третьей итерации

04

Проверки

Рабочая БД с тестовыми данными и 16 запросов к ней — все прошли, включая негативные

ШАГ 1 · СКОЛЬКО БАЗ ДАННЫХ

Два хранилища — по характеру данных

SQL · ОСНОВНАЯ БД

PostgreSQL 16

Все доменные данные: пользователи, трекеры, записи, цели, подписки, платежи, AI-сводки, уведомления. Единственный источник истины — данные живут постоянно

КРИТЕРИЙ
потеря недопустима

СВЯЗИ
FK + транзакции

NOSQL · ЭФЕМЕРНОЕ СОСТОЯНИЕ

Redis

Magic-link токены (TTL 15 мин), rate limiting, очередь фоновых задач AI и уведомлений, краткоживущий кэш. Потеря равносильна повторной авторизации

КРИТЕРИЙ
естественный TTL

ОПЕРАЦИИ
O(1) в памяти

Критерий разделения простой: если на данные ссылаются другие данные и их потеря недопустима — они в PostgreSQL; если данные технические и краткоживущие — в Redis. Это разделение уже заложено в контейнерную архитектуру четвёртой итерации.

ШАГ 1 · ОБОСНОВАНИЕ

Почему не одна и не три

АНТИПРИМЕР 1

Очередь задач в PostgreSQL

Постоянные INSERT/DELETE по горячей таблице: конкуренция за блокировки, рост вакуума — деградация именно тех операций, где требование p95 < 300 мс. Redis делает это за O(1) в памяти

АНТИПРИМЕР 2

Доменные данные в Redis

Теряем транзакции, ссылочную целостность и durability: платёж нельзя хранить в хранилище, которое может быть очищено вытеснением по LRU

АНТИПРИМЕР 3

Третья СУБД «про запас»

При 10 000 DAU отдельный документный стор или колоночная БД дадут только сложность: бэкапы, мониторинг, согласованность. Полуструктурированные данные покрывает JSONB

Когда пересмотрим: при росте нагрузки на порядок первый кандидат на вынос — аналитика (read replica или колоночное хранилище). Доменное ядро останется реляционным.

ШАГ 2 · SQL ИЛИ NOSQL

Пять аргументов за реляционную модель

01	**Связи — предмет инвариантов.** Значение поля должно принадлежать тому же трекеру, что и запись. В SQL — внешний ключ и один триггер; в NoSQL — код приложения, который можно обойти
02	**Транзакции по нескольким строкам.** Сохранение записи со значениями (UC-01) и оплата с активацией подписки (UC-04) атомарны только на ACID
03	**Запросы — агрегирующие.** Календарь месяца, средние по неделям, прогресс целей, выручка — это JOIN, GROUP BY и оконные функции, домашняя территория SQL
04	**Схема уже формализована.** Проектная диаграмма классов даёт 13 сущностей и 9 перечислений — контракт, а не «документы свободной формы»
05	**152-ФЗ на уровне СУБД.** Row-level security: пользователь физически не прочитает чужие строки даже при ошибке в коде приложения

ШАГ 2 · ОСОЗНАННЫЙ ГИБРИД

NoSQL-островки внутри PostgreSQL

JSONB

constraints полей

Ограничения разнотипны: min/max для шкалы, options для выбора, unit для числа. Читаются целиком, по ним не ищут — четыре реляционные таблицы не дали бы ничего

JSONB

insights + advice

Инсайты и советы AI-сводки — список строк переменной длины, возвращаемый моделью qwen2.5. Хранится и отдаётся массивом, разбирается jsonb_array_elements_text

EAV-LITE

value TEXT для 6 типов

Значение хранится строкой, как в классе «ЗначениеПоля». Тип контролирует конструктор полей, а аналитика приводит к numeric на лету — без шести таблиц значений

Принцип: реляционное ядро — там, где нужны целостность и агрегации; документные островки — там, где данные атомарны для чтения. Гибрид даёт оба качества без второй СУБД.

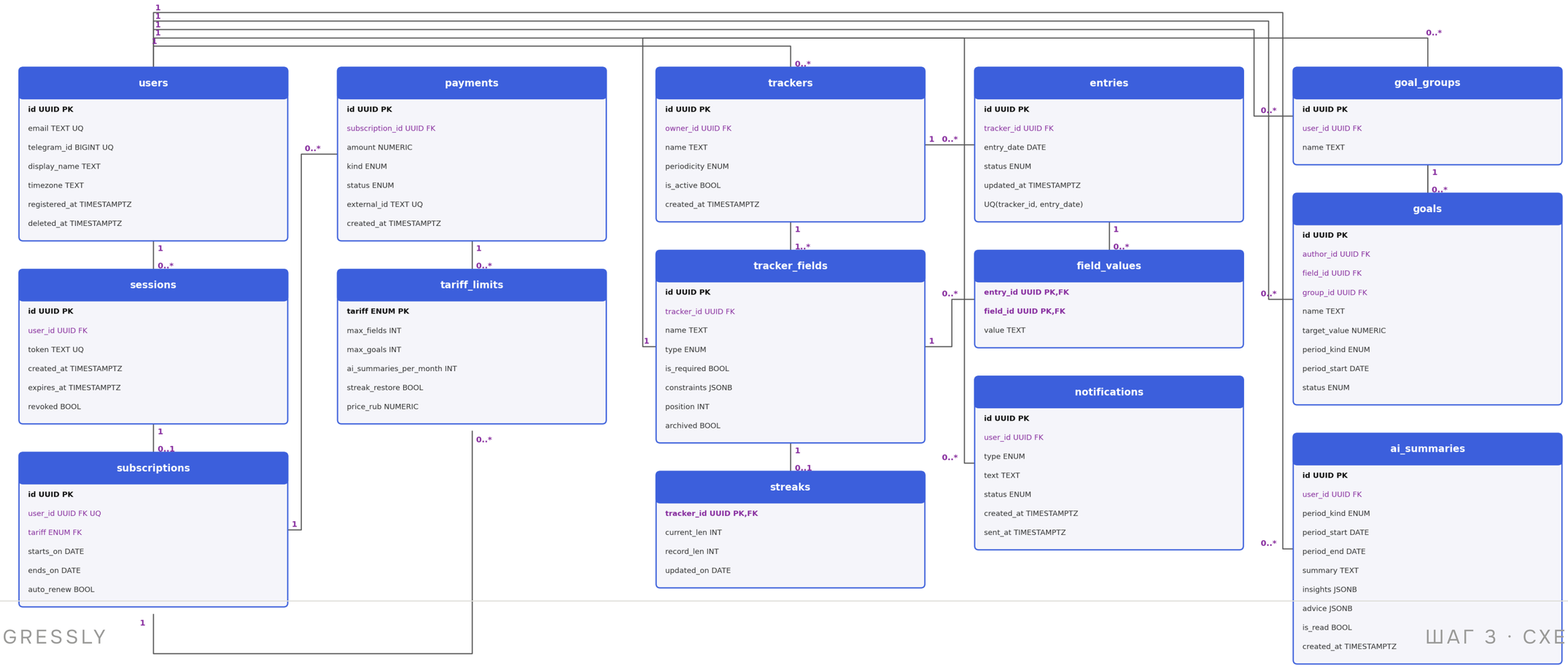
ШАГ 3 · СХЕМА ИЗ ДИАГРАММЫ КЛАССОВ

ORM-маппинг: класс → таблица

КЛАСС (ИТЕРАЦИЯ 3)	ТАБЛИЦА	ПРАВИЛО ОТОБРАЖЕНИЯ
Пользователь	`users`	идПользователя: UUID → первичный ключ
Сессия	`sessions`	«владелец 1—0..*» → FK + индекс
Подписка	`subscriptions`	кратность «1—0..1» → UNIQUE(user_id)
Платёж	`payments`	статус «= СОЗДАН» → DEFAULT 'CREATED'
Трекер	`trackers`	«активен = истина» → DEFAULT TRUE
ПолеТрекера	`tracker_fields`	композиция 1—1..* → FK ON DELETE CASCADE
Запись	`entries`	«за конкретную дату» → UNIQUE(tracker_id, entry_date)
ЗначениеПоля	`field_values`	композиция с двух сторон → составной PK
Стрик	`streaks`	«трекер 1—0..1» → PK по tracker_id
Цель / ГруппаЦелей	`goals, goal_groups`	три ассоциации Цели → три внешних ключа
AI-сводка	`ai_summaries`	Список<Строка> инсайтов и советов → JSONB
Перечисления (9 шт.)	`CREATE TYPE … AS ENUM`	«enumeration» → ENUM-тип PostgreSQL

ШАГ 3 · СТРУКТУРА

ER-диаграмма: 14 таблиц, 4 группы



ШАГ 3 · ЦЕЛОСТНОСТЬ НА УРОВНЕ БД

Инварианты, которые держит схема

UNIQUE

Одна запись на дату

UNIQUE(tracker_id, entry_date) — физически невозможно создать две записи трекера на один день

TRIGGER

Поле своего трекера

Триггер отклоняет значение, если поле и запись принадлежат разным трекерам

CHECK

Рекорд ≥ текущего

CHECK(record_len ≥ current_len) — стрик не может превысить рекорд

UNIQUE

Идемпотентность оплаты

payments.external_id UNIQUE — повторный webhook YooKassa не создаст дубль платежа

CASCADE

Удаление аккаунта

FR-04 выполняется одним DELETE на users: все композиции каскадны

RLS

Изоляция по 152-ФЗ

Политики на trackers, entries, field_values: роль приложения видит только строки владельца

ШАГ 4 · НАПОЛНЕНИЕ

Согласованный тестовый датасет

5

пользователей

На трёх тарифах: FREE, FREE_PRO (пробный), PRO — с подписками и сессиями

6

трекеров · 16 полей

Все шесть типов полей: число, шкала, флаг, текст, выбор, время

106

записей за 60 дней

С честными пропусками и частичными заполнениями — стрики сходятся с пересчётом

386

значений полей

Детерминированные значения от даты: датасет воспроизводим в любой день

7

платежей

Все статусы YooKassa: создан, оплачен, отклонён; первичные и автосписания

3+6

AI-сводки и цели

Сводки с инсайтами и советами в JSONB, цели в статусах ACTIVE и ACHIEVED

ШАГ 4 · РАБОТОСПОСОБНОСТЬ

16 проверок — все прошли

	СЦЕНАРИЙ	ИТОГ
Q1	UC-01: форма трекера за день одним запросом	**OK**
Q2	UC-01: идемпотентное сохранение записи	**OK**
Q3	Календарь месяца со статусами дней	**OK**
Q4	Стрик 13 и рекорд 14 совпали с пересчётом	**OK**
Q5	Аналитика по неделям из сырых значений	**OK**
Q6	Drill-down по полю «Настроение»	**OK**
Q7	Прогресс целей через представление	**OK**
Q8	UC-02: доступ к AI по тарифу и лимиту	**OK**

	СЦЕНАРИЙ	ИТОГ
Q9	UC-02: сводка с инсайтами из JSONB	**OK**
Q10	UC-04: платежи и статус подписки	**OK**
Q11	Админ: тарифы, выручка, конверсия	**OK**
Q12	Retention: активность за 7 дней	**OK**
Q13	Конструктор полей: ограничения JSONB	**OK**
Q14	Триггер отклонил чужое поле (ERROR)	**ERROR***
Q15	UNIQUE отклонил дубль записи (ERROR)	**ERROR***
Q16	RLS: Мария видит 10 записей вместо 107	**OK**

*** Ожидаемые ошибки** — негативные тесты: база сама отклоняет некорректные данные. Целостность контролируется на уровне СУБД, а не только приложения.

ИТОГ ИТЕРАЦИИ

База спроектирована, наполнена и проверена

- **Два хранилища:** PostgreSQL — источник истины, Redis — эфемерное состояние
- **SQL с JSONB-островками:** целостность и агрегации там, где они нужны
- **14 таблиц** по ORM-маппингу проектной диаграммы классов
- **16/16 проверок** на рабочей базе, включая негативные и RLS

Артефакты: README.md · ER-диаграмма.png · db/schema.sql · db/seed.sql · db/queries.sql · db/docker-compose.yml · Результаты проверок.md

```
# воспроизвести базу одной командой
docker compose -f db/docker-compose.yml up -d

# прогнать проверки
psql -U postgres -d progressly -f db/queries.sql
```